

오늘 대시보드: 기능과 흐름

2026-10-02 · 내 맥에서만 도는 개인 대시보드 (웹 화면 + 터미널 화면)

목차

1. 한눈에
2. 구조와 설계 의도
3. 화면과 기능
4. 사용 흐름
5. 데이터 흐름
6. AI 생성 흐름
7. 실행
8. 파일 구성
9. 부록

그림

- 그림 1. 전체 구조
- 그림 2. 하루와 한 주 흐름
- 그림 3. AI 생성 순서

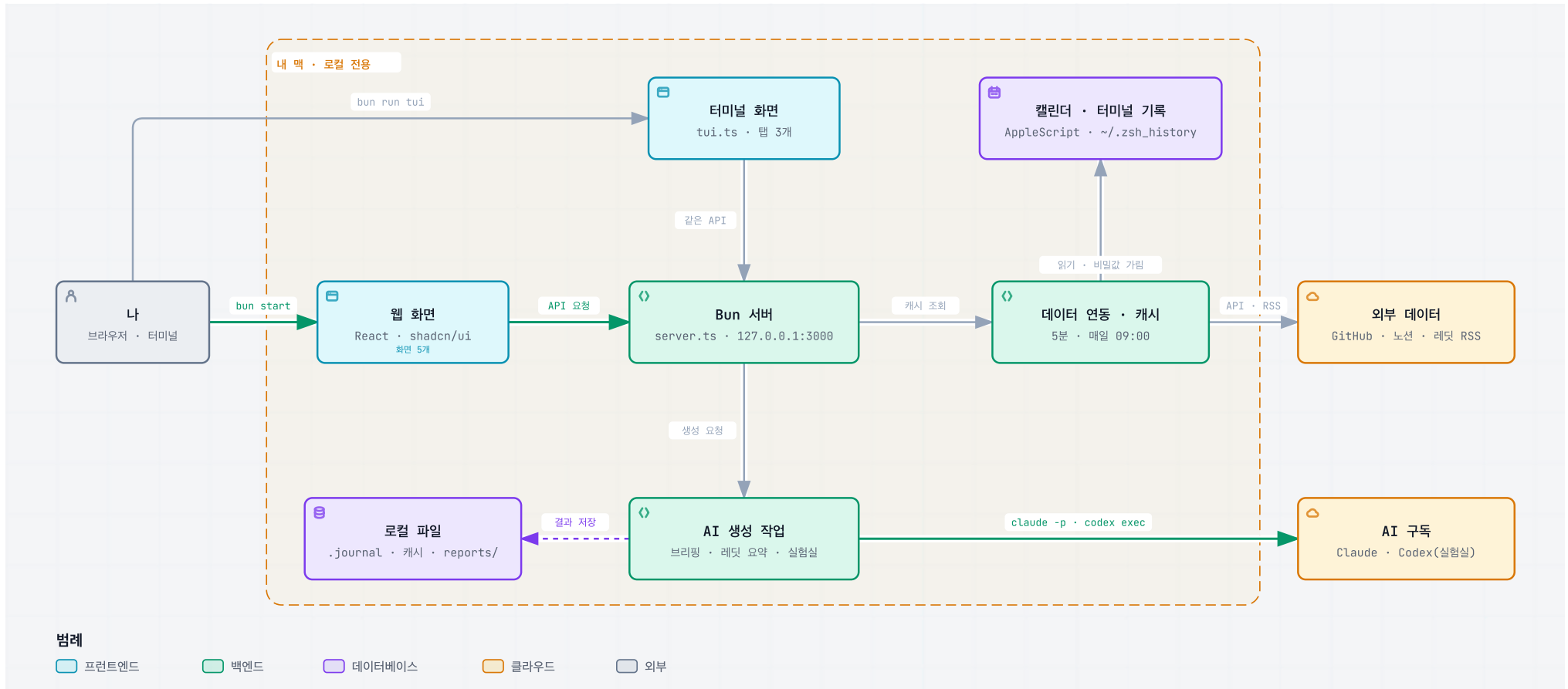
1. 한눈에

오늘 일정, 노션 할 일, GitHub 활동, 터미널 작업을 한곳에 모아서 하루가 얼마나 진행됐는지 보여 주는 대시보드다. 레딧의 AI 개발·기술·모델 서브레딧에서 오늘 인기 글을 매일 모아 한국어로 옮기고 순위를 매기며, 로컬 Claude Code가 아침 브리핑, 저녁 하루 마무리, 금요일 주간 업무 리포트, 오픈소스 인사이트를 만든다. 모델 실험실에서는 공개 검증 문항과 내 업무 과제로 Claude와 Codex(GPT) 모델을 비교한다.

- 웹 화면(`bun start`)과 터미널 화면(`bun run tui`) 두 가지로 쓴다.
- 서버는 `127.0.0.1`에서만 열고 배포하지 않는다.
- AI는 API 키 없이 구독 로그인으로 돌린다. Claude 구독을 쓰고, 모델 실험실만 ChatGPT 구독도 쓴다.
- 웹 화면은 React와 shadcn/ui로 만들었고 macOS의 라이트·다크 설정을 따른다.

화면, 서버, 데이터 출처, AI가 어떻게 연결되는지는 그림 1에 정리했고, 이렇게 짠 이유는 2장에서 설명한다.

그림 1 전체 구조



웹 화면(`bun start`)과 터미널 화면(`bun run tui`)은 같은 Bun 서버 API를 쓴다. 서버는 맥 안의 캘린더·터미널 기록과 외부의 GitHub·노션·레딧에서 데이터를 모아 캐시하고, AI 생성 작업은 Claude Code(`claude -p`)와 모델 실험실용 Codex(`codex exec`)를 불러 결과를 로컬 파일에 저장한다. 점선 상자 안은 모두 이 맥에서 돌고, 밖으로 나가는 호출은 공식 API·RSS와 구독 로그인뿐이다.

2. 구조와 설계 의도

이 대시보드는 한 사람이 하루를 관리하려고 쓰는 도구다. 작업 중에 흠뻑 보고 다시 일로 돌아가는 화면이라서, 세 가지 질문에 바로 답하도록 짰다. 오늘 무엇을 해야 하고 얼마나 했나, 요즘 내 분야에서 볼 것은 무엇인가, 이 둘을 AI가 요약하고 우선순위를 매기면 어떤가. 구조를 정하기 전에 원칙 세 가지를 먼저 세웠고, 나머지 결정은 이 원칙과 데이터 출처마다 다른 속도·한도에 맞췄다.

원칙: 로컬, 공식 경로, 구독

- **내 맥 안에서만 돈다.** 일정, 터미널 기록, 노션 할 일에는 개인 정보와 회사 정보가 섞여 있다. 그래서 서버는 127.0.0.1에만 열고 배포하지 않는다. 업무에 쓰고 공유할 도구라서 데이터 위험이 낮고 업무 가치가 분명한 기능만 넣었다. Slack은 워크스페이스 관리자 승인이 필요하고, Gmail은 회사 메일을 외부 AI로 보내게 되어 검토 후 뺐다.
- **공식 경로만 쓴다.** 공식 API, 공식 RSS, 맥 안의 데이터만 읽고 크롤링은 하지 않는다. X는 무료 API가 없고 약관이 수집을 막아서 뺐다. 레딧은 로그인 없이 쓸 수 있는 공식 RSS만 쓴다(.json 주소는 403, 공식 Data API는 승인이 필요하다).
- **API 키 없이 구독으로 돌린다.** AI는 로컬 claude -p (Claude 구독)로, 실험실의 GPT 모델은 codex CLI(ChatGPT 로그인)로 부른다. 서버는 AI를 부를 때 ANTHROPIC_API_KEY를 지워서 사용량 과금으로 넘어가지 않게 막는다.

서버 한 곳에서 모으고, 가진 값을 먼저 보여 준다

웹 화면과 터미널 화면은 각자 데이터를 모으지 않고 같은 Bun 서버 API를 쓴다(그림 1).

- **서버를 하나로 둔 이유:** 같은 사람이 두 화면을 오가며 쓰므로 숫자, 상태 기호(○ ● ●), 단축키가 같아야 다시 배울 것이 없다. 터미널 화면은 서버가 꺼져 있으면 알아서 켜다. 웹 화면도 같은 서버가 묶어서 내보내므로 화면용 서버를 따로 띄우지 않는다.
- **캐시를 둔 이유:** 출처마다 속도와 한도가 크게 다르다. 캘린더 두 달 치를 AppleScript로 읽는 데 약 40초가 걸리고, GitHub 검색은 1분에 30번, 레딧 RSS는 로그인 없이 1분에 1번까지다. 그래서 서버는 가진 값을 바로 주고 새 값은 뒤에서 받는다. 캐시는 파일로 남겨서 서버를 다시 켜도 한도를 다시 쓰지 않는다.
- **느린 출처가 전체를 막지 않게:** 노션 요청은 15초가 지나면 끊는다. 노션이 느려도 캘린더 전체가 멈추지 않는다.
- **캐시 파일을 기능마다 나눈 이유:** 예전 코드로 돌던 서버가 새 기능의 결과를 덮어쓴 일이 있었다. 레딧 결과는 .reddit-cache.json에 따로 뒤서 같은 일이 생기지 않게 했다.

쓰기는 최소로, 바깥 입력은 믿지 않는다

- **노션에는 두 가지만 쓴다.** 할 일 추가와 상태 변경이다. 할 일의 원본은 노션에 두고 대시보드는 보고 체크하는 창구로 쓴다. 쓴 결과는 캐시에 바로 반영하고(다시 읽으면 40초), 느린 캘린더 읽기가 나중에 도착해도 방금 쓴 내용이 덮이지 않게 다시 적용한다.
- **체크는 화면에 먼저 반영한다.** 노션 저장이 실패하면 원래대로 되돌리고 이유를 보여 준다. 저장 중인 항목은 다시 눌러도 요청을 또 보내지 않는다.
- **쓰기 요청은 JSON으로만 받는다.** 노션 쓰기, 레딧 수집, 실험 시작이 그렇다. 다른 웹사이트가 로컬 서버에 몰래 쓰는 것을 막기 위해서다. 다른 출처에서 보내는 JSON 요청은 브라우저의 사전 확인 단계에서 막힌다.
- **터미널 기록의 비밀값을 가린다.** 이 기록은 화면, 보고서 캡처, Claude 프롬프트로 나가기 때문이다.
- **레딧 글은 도구를 끈 Claude로만 요약한다.** 누구나 쓸 수 있는 글이라 믿을 수 없는 입력으로 다룬다.

AI는 백그라운드 작업으로, 결과는 정해진 JSON으로

- **백그라운드 작업과 상태 확인:** `claude --effort max`는 몇 분이 걸려 HTTP 요청 제한(255초)을 넘을 수 있다. 그래서 서버가 작업을 뒤에서 돌리고, 화면은 만드는 동안 1.5초마다 상태를 묻는다(그림 3).
- **글이 아니라 JSON:** 화면이 결과를 카드, 링, 막대로 그리려면 필드가 정해져 있어야 한다. 흠뻑 보는 화면에 긴 글 덩어리는 맞지 않았다.
- **이전 결과를 지우지 않는다:** 만드는 1~4분 동안과 실패했을 때도 이전 결과를 흐리게 남긴다. 빈 화면 대신 결과 없음, 만드는 중, 실패, 날짜 지남 상태를 각각 보여 준다.
- **보고용 글은 사실만 쓴다:** 주간 리포트 프롬프트는 "데이터에 있는 사실만 쓰고 과장하지 말라"고 정해 둔다.

하루와 한 주를 잇는 고리

아침 브리핑이 계획을 세우고, 평일 18:00 하루 마무리가 그 계획을 실제 활동(일정, 노선 할 일, GitHub, 터미널)과 비교한다. 못 끝낸 일은 `.journal.json`을 거쳐 다음 날 브리핑에 먼저 들어가고, 금요일 18:30 주간 리포트가 한 주의 기록을 모아 파일로 남긴다(그림 2). 계획이 계획으로 끝나지 않고, 못 한 일이 사라지지 않고, 주간 보고를 따로 쓰지 않아도 되게 하려는 구조다.

- 자동 작업은 서버가 켜져 있고 맥이 깨어 있을 때만 돈다. 그래서 09:00 수집을 놓쳤으면 다음에 열 때 뒤에서 모으고(실패 직후 30분은 다시 시도하지 않는다), 놓친 브리핑과 사무리는 화면에서 직접 만든다.
- 흠의 진행률 링은 내가 직접 체크하는 노선 할 일의 완료 비율로 센다. 체크하면 바로 바뀐다.

한쪽이 독식하지 않게 모은다

- **오픈소스:** 스타 수로만 줄 세우면 큰 LLM 레포가 웹툰·만화 같은 작은 분야를 묻어 버렸다. 그래서 관심 가중치 × (인기 + 성장 속도)로 점수를 매기고, 분야마다 상위 3개는 꼭 보여 준다.
- **레딧:** 여러 서브레딧을 한 요청으로 묶으면 큰 곳이 40개 중 38개를 차지했다. 그래서 서브레딧마다 따로 5개씩 받는다. 1분에 1곳만 받을 수 있어 10곳이면 약 10분이 걸리므로 하루 한 번(09:00) 모은다.

모델 실험실: 믿을 수 있게 비교한다

처음에는 직접 만든 업무 과제 4개를 Opus 혼자 채점했다. 과제가 적고 채점자도 참가 모델이라 결과가 치우칠 수 있어서 방식을 바꿨다.

- **공개 검증 문항을 고정해 쉰다.** KMMLU, HRM8K, IFEval에서 시드를 정해 한 번 뽑아 두고 매번 같은 문항을 푼다. KMMLU-Pro는 접근 제한과 비상업 라이선스 때문에 뺐다.
- **계열이 다른 채점단을 둔다.** Sonnet과 GPT-6 Sol이 이름을 가린 채 따로 채점하고 평균을 낸다. 실제로 두 채점자 모두 자기 모델 답에 1점 남짓 더 줬다(Sonnet +1.3, Sol +1.2).
- **신뢰구간과 짝 비교를 함께 본다.** 작은 점수 차를 우열로 읽지 않기 위해서다. 9월 30일 1·2위(Astra 91.6, Opus 91.0)는 "차이 없음"이다.
- **조건을 맞춘다.** Claude와 Codex 모두 도구를 끄고, 사용자 설정과 세션 기록 없이 한 번에 답하게 했다. 이름을 가린 내 선택과 채점단 1위의 일치율로 채점단도 점검한다.
- **구독 사용량을 조절한다.** 빠른 실험(33회 호출)과 정밀 실험(87회 호출)으로 나눴다.

화면: 배치는 지키고 부품을 바꿨다

웹 화면은 마지막에 React와 shadcn/ui로 다시 만들었다(ui/ 폴더). 서버의 데이터 수집과 API는 그대로 두고 화면만 바꿨다.

- **시안 두 개를 같은 데이터로 비교했다.** shadcn/ui 위에 방향이 다른 시안 두 개를 만들고, 같은 기능과 같은 데이터로 나란히 비교한 뒤 1안을 골랐다. 데이터를 불러오고 쓰고 상태를 계산하는 코드를 화면과 분리해(ui/src/shared/) 두 시안이 함께 썼기 때문에 가능했다.
- **배치는 유지했다.** 카드, 진행률 링, 막대로 묶은 배치가 훑어 보는 용도에 맞았다. 카드와 링을 걷어 내고 표 위주로 만든 첫 시안은 디자인 요소가 사라져서 쓰지 않았다. 그래서 배치와 색은 그대로 옮기고 버튼, 체크박스, 탭, 대화상자 같은 부품만 shadcn/ui로 바꿨다.
- **화면을 늘리지 않고 한 화면 안에서 더 본다.** 목록의 줄을 누르면 옆에서 상세 창이 열린다. 할 일 필터, 레딧 검색과 정렬, 캘린더 구분 필터, 실험실 표 정렬도 같은 생각으로 더했다.
- **흐름은 한 화면에 들어오게 줄였다.** 훑어 보는 화면이라 스크롤이 없는 쪽이 낫다. 글자 규격은 그대로 두고 전체 크기만 93.75%로 줄였고, 흐름의 여백과 목록 개수를 줄였다. 활동이 많은 날에는 아직 조금 넘친다(알려진 한계).
- **글꼴은 Pretendard 하나로, 크기는 역할별 규격으로 정했다.** 제목, 본문, 버튼, 날짜 같은 역할마다 크기와 굵기를 정해 두어(ui/styles/type.css) 화면마다 따로 맞추지 않는다.
- **움직임은 상태가 바뀔 것을 알릴 때만 쓴다.** 링과 숫자, 막대는 처음 열 때와 값이 바뀔 때만 움직이고, 화면을 오갈 때마다 다시 재생하지 않는다. macOS에서 '동작 줄이기'를 켜면 움직임을 끈다.
- **정보가 없는 꾸밈은 뺐다.** 같은 이름이 다른 숫자를 가리키지 않게 진행률 이름을 나눴다(흐름 노선 완료 비율, AI 브리핑은 AI가 추정한 값). 인사말 제목, 반짝이 아이콘, 가운데점으로 길게 이은 설명 줄은 없애고, 덜 중요한 정보는 마우스를 올리면 보이게 했다.
- **상태 화면을 바로 확인한다.** 주소에 ?state=loading, ?state=empty, ?state=error 를 붙이면 불러오는 중, 데이터 없음, 오류 화면이 열린다. 실제 데이터를 건드리지 않고 확인하려는 것이라 이때는 저장과 실행 요청을 보내지 않는다.

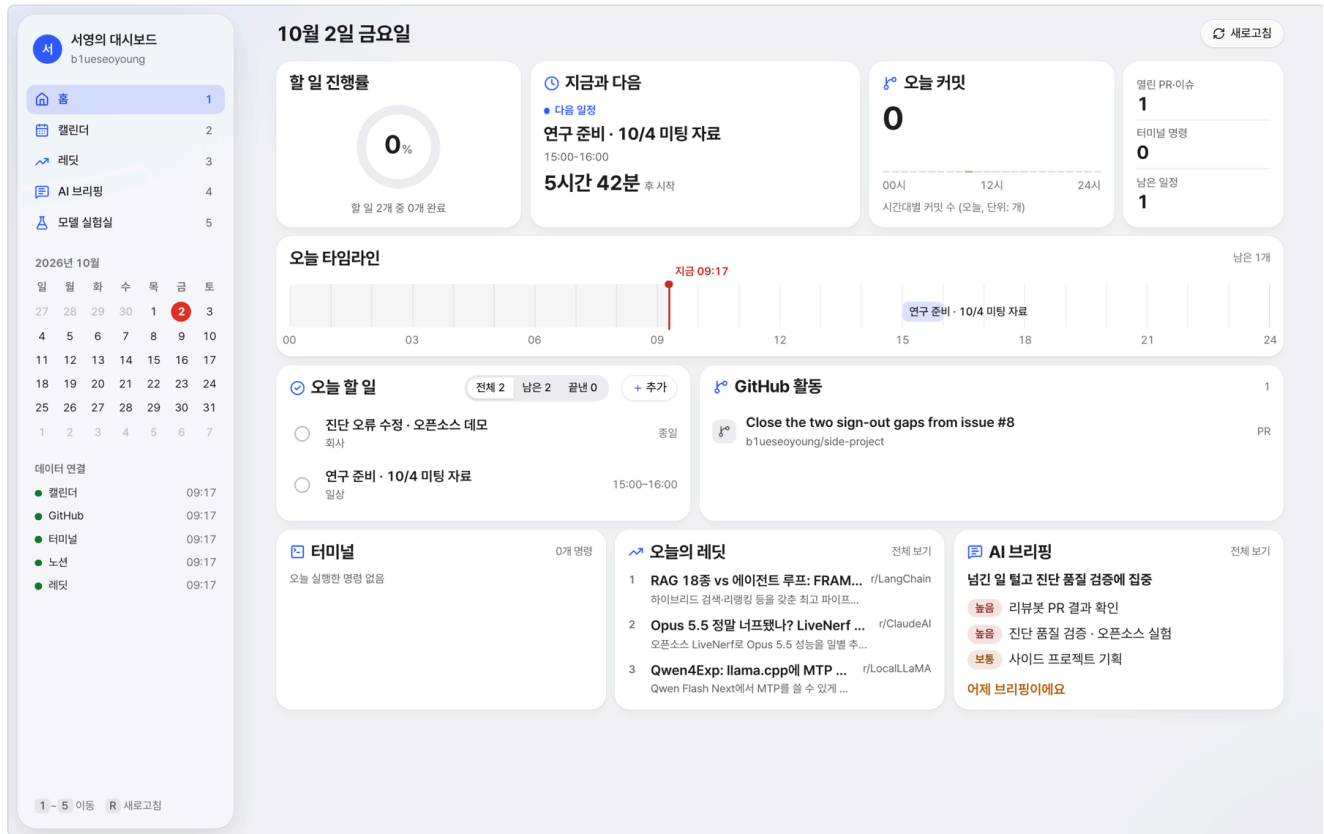
알려진 한계

- AppleScript는 반복 일정의 뒤 회차를 놓칠 수 있다. 문제가 되면 Swift EventKit로 바꾼다.
- 비밀값 가리키는 패턴 기반이라 모든 형식을 잡지는 못한다. 새 형식이 보이면 패턴을 더한다.
- 오픈소스 점수는 경험적 공식이다. 필요하면 Claude 평가로 바꾼다.
- 실험실은 문항 수가 적고 채점자도 참가 모델이라 결과는 참고용이다.
- 터미널 화면에는 하루 마무리, 주간 리포트, 모델 실험실이 없다. 웹 화면에서 본다.
- 줄이 상세 창으로 이어지는 화면 전환 같은 새 움직임은 아직 흐름에만 있다.
- 휴대폰에는 마우스를 올리는 동작이 없어서, 마우스를 올려야 보이게 옮긴 정보는 휴대폰에서 보이지 않는다.
- 흐름은 일정과 GitHub 활동이 많은 날 한 화면을 조금 넘는다(1728×1000 창에서 10월 1일 오후에는 85~111px 스크롤, 활동이 적은 10월 2일 아침에는 넘치지 않음).

3. 화면과 기능

화면은 홈, 캘린더, 레딧, AI 브리핑, 모델 실험실 다섯 개다. 사이드바나 숫자 키 1~5로 이동한다.

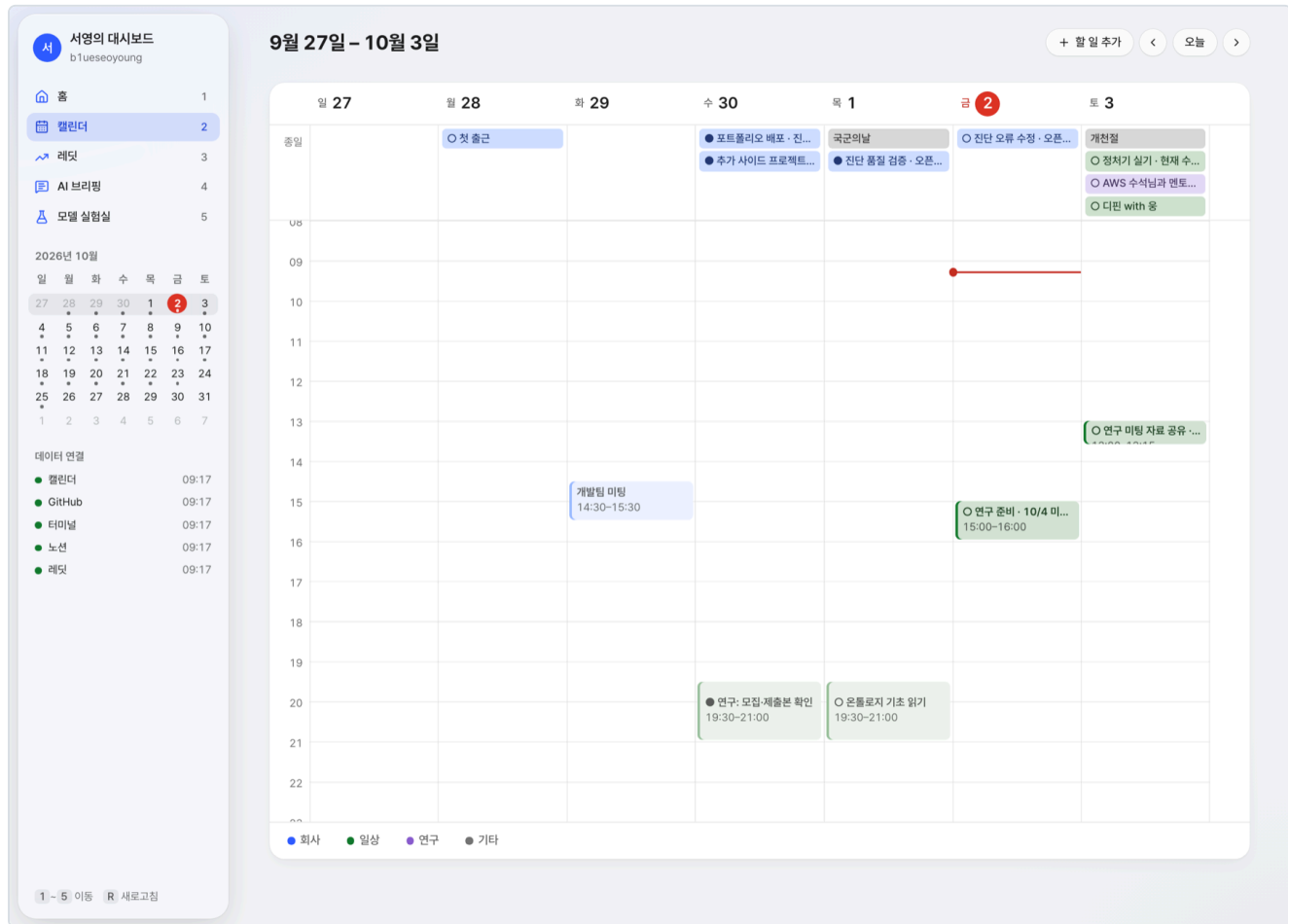
홈



홈 화면

- 날짜 제목과 할 일 진행률 링(오늘 노선 할 일 중 완료한 비율. 종일 할 일도 포함하고, 체크하면 바로 바뀜)
- 지금과 다음: 진행 중인 일정이나 다음 일정까지 남은 시간, 그 뒤에 오는 일정 2개
- 오늘 커밋 수와 시간대별 막대, 열린 PR·이슈 수, 터미널 명령 수, 남은 일정 수
- 24시간 타임라인: 일정 막대와 빨간 "지금" 선. 진행 중인 일정은 지난 만큼 채워짐. 제목이 잘리거나 다음 일정에 가려지는 일정은 타임라인 아래에 시각과 제목으로 최대 3개까지 따로 보임
- 오늘 할 일: 노선 Task DB에서 오늘 '일정'인 할 일. 동그라미를 누르면 노선에서 완료로 바뀌고, 줄을 누르면 상세 창(날짜, 시간, 구분, 상태, 중요도, 노선에서 열기)이 열림. 할 일이 둘 이상이면 전체·남은·끝낸으로 걸러 봄. "추가"로 새 할 일을 넣음(노선에 바로 저장)
- GitHub 활동(5개, 나머지는 "더 보기"), 자주 쓴 터미널 명령 5개와 최근 명령 2개, 오늘의 레딧(먼저 볼 글 3개), AI 브리핑 요약과 하루 마무리 달성률
- 레딧 카드와 AI 브리핑 카드를 누르면 해당 화면으로 이동

캘린더



캘린더 화면

- 주간 시간표: 종일 일정 줄, 시간 칸, 오늘의 "지금" 선
- 이전 주, 다음 주, 오늘 버튼. 방향키 ← →도 된다
- 사이드바 미니 캘린더에서 날짜를 누르면 그 주로 이동
- 일정 색은 구분을 따름: 회사 파랑, 일상 초록, 연구 보라, 기타 회색. 아래 범례를 누르면 그 구분을 숨기거나 다시 보임
- 일정을 누르면 상세 창이 열림
- 노선 할 일도 함께 보임. 시간이 있으면 시간 칸, 날짜만 있으면 종일 줄에 상태 기호(○ 시작 전, ● 진행 중, ● 완료)와 함께 표시. 같은 날 같은 제목이 구글 캘린더에도 있으면 하나만 보임
- "할 일 추가" 버튼이나 요일 머릿글을 눌러 그 날짜에 노선 할 일을 추가(제목, 날짜, 구분, 시작·끝 시간). '종일'이 기본으로 켜져 있고, 끄면 시작·끝 시간을 정함
- 휴대폰에서는 날짜별 목록으로 바뀜

레딧

서영의 대시보드
bTuesoyoung

홈

캘린더

레딧

AI 브리핑

모델 실험실

2026년 10월

일	월	화	수	목	금	토
27	28	29	30	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

데이터 연결

캘린더

GitHub

터미널

노션

레딧

오늘의 레딧 AI 핫 뉴스

오늘 09:17 수집

모델 열화 논쟁과 에이전트 중심 설계가 부상

1분 전 요약

1 모델 열화-한도 불만

Opus 5.5 너프 의혹과 LiveNerf 추적, Codex 한도 불만, 토큰 절감 도구가 동시에 올라왔다.

2 차세대 모델 공개 임박

Google의 내부 최상위 모델과 Gemini 4 Argon 논의, Qwen MTP 지원 등 새 모델 소식이 이어졌다.

3 RAG보다 에이전트 루프

FRAMES 벤치마크에서 에이전트 루프가 최고 RAG 파이프라인을 앞섰고, 도구 호출 방식 개선 논의도 나왔다.

1 RAG 18종 vs 에이전트 루프: FRAMES에서 78.9% 대 92.7%

하이브리드 검색·리랭킹 등을 갖춘 최고 파이프라인보다 에이전트 루프가 높은 점수를 냈다.

LangChain 1위 연구 10시간 전

RAG 18종 대비 에이전트 루프가 92.7%로 앞선 실측 비교라 설계 판단에 바로 쓸 수 있다.

2 Opus 5.5 정말 너프 됐나? LiveNerf 8일차

오픈소스 LiveNerf로 Opus 5.5 성능을 일별 추적한 업데이트다.

Claude 5위 도구 1시간 전

Opus 5.5 성능 저하 의혹을 오픈소스로 일별 추적해 모델 품질 변동을 점검할 수 있다.

3 Qwen4Exp: llama.cpp에 MTP 추가

Qwen Flash Next에서 MTP를 쓸 수 있게 됐고 PR은 17시간 만에 병합됐다.

오픈 모델 3위 도구 13시간 전

llama.cpp에 Qwen MTP가 병합돼 로컬 추론 속도와 모델 선택에 영향을 준다.

4 도구가 늘수록 에이전트가 오 호출... 코드로 호출하게 하라

도구 스키마가 컨텍스트를 채우는 문제를 모델이 코드를 써서 도구를 호출하는 방식으로 푼다.

에이전트 4위 실전 10시간 전

도구가 많아질 때 오호출을 줄이는 코드 기반 호출 방식을 다룬다.

5 사용자 오류엔 버티는 LLM, '검 증된 출처'엔 틀린 답도 수용

같은 틀린 주장도 검증된 출처로 포장하면 모델이 답을 바꾼다는 NeurIPS 2026 연구다.

ML 연구 3위 연구 10시간 전

검증된 출처 표기에 LLM이 쉽게 틀린 답을 수용한다는 NeurIPS 2026 연구다.

전체 49

오픈 모델 5 AI 뉴스 5 OpenAI 4 Claude 5 ML 연구 5 이미지·영상 모델 5 LangChain 5 RAG 5 에이전트 5 AI 코딩 5

글 49개

Q 제목이나 요약에서 찾기

서브레딧별 순위

1 Clef: Cloudflare의 오픈 웨이트 의사결정 모델

Cloudflare가 Clef라는 오픈 웨이트 decision 모델을 공개한 것으로 보인다.

오픈 모델

2 286 Tandy 1000에서 채팅과 이미지 생성 돌리기

DOS 프로그램 DeskMind가 WiFi로 PC의 Python 서버를 거쳐 Qwen3.8-27B를 구동한다.

오픈 모델

3 Qwen4Exp: llama.cpp에 MTP 추가

Qwen Flash Next에서 MTP를 쓸 수 있게 됐고 PR은 17시간 만에 병합됐다.

오픈 모델

4 Heretic, PewDiePie에 등장

Heretic이 유튜브비 PewDiePie 영상에서 다뤄졌다는 소식이다.

오픈 모델

레딧 화면(윗부분)

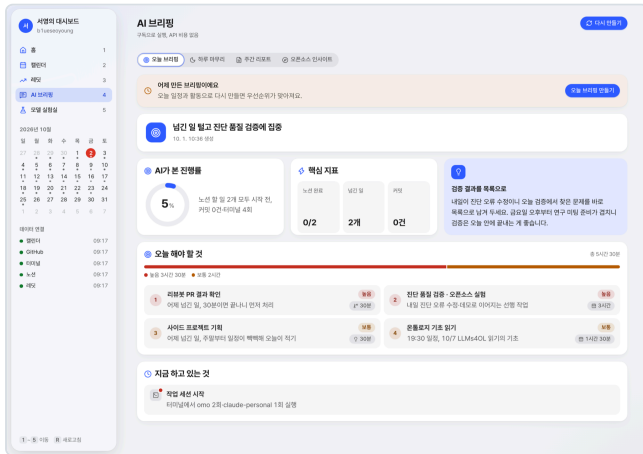
- AI 개발·기술·모델 서브레딧 10곳(LocalLLaMA, singularity, OpenAI, ClaudeAI, MachineLearning, StableDiffusion, LangChain, Rag, AI_Agents, ChatGPTCoding)에서 오늘 상위 글을 곳마다 5개까지 모음. 목록과 한국어 이름은 `reddit.json`에서 바꿈
- Claude(sonnet)가 모든 글에 한국어 제목과 한 줄 요약을 붙이고, 오늘의 흐름 3가지와 먼저 볼 TOP 5(이유 포함)를 고름. 레딧 RSS에는 추천 수가 없어서 순위는 서브레딧 안 오늘 순위와 AI 개발자에게 중요한 정도로 정함
- TOP 5 카드를 누르면 레딧 원문이 열림. 아래 목록은 서브레딧 탭으로 거르고, 제목·요약 검색과 정렬(서브레딧별 순위, 최신 순, 순위 높은 순)을 할 수 있음
- 목록의 글을 누르면 상세 창이 열림: 요약, 먼저 볼 이유, 서브레딧과 순위, 글쓴이, 영어 원문 제목, 본문 앞부분, "레딧에서 열기"
- 매일 09:00에 자동으로 모으고, "새로 가져오기"로 바로 모을 수도 있음. 로그인 없이는 1분에 1곳만 받을 수 있어 10곳이면 약 10분 걸림. 그동안 이전 결과를 그대로 보여 주고 진행 배너만 띄움
- 가져오는 중, 한국어로 옮기는 중, 가져오기 실패, 요약 실패, 아직 없음, 어제 모은 글일 때 화면이 각각 따로 있음

오늘 대시보드: 기능과 흐름

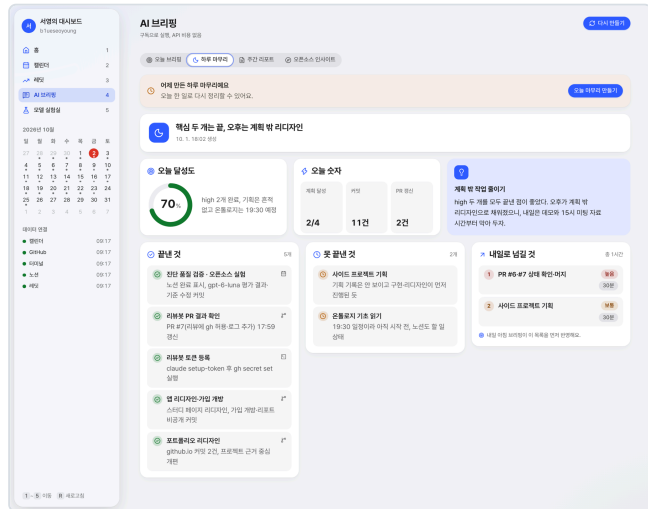
8

AI 브리핑

템은 오늘 브리핑, 하루 마무리, 주간 리포트, 오픈소스 인사이트 네 개다.



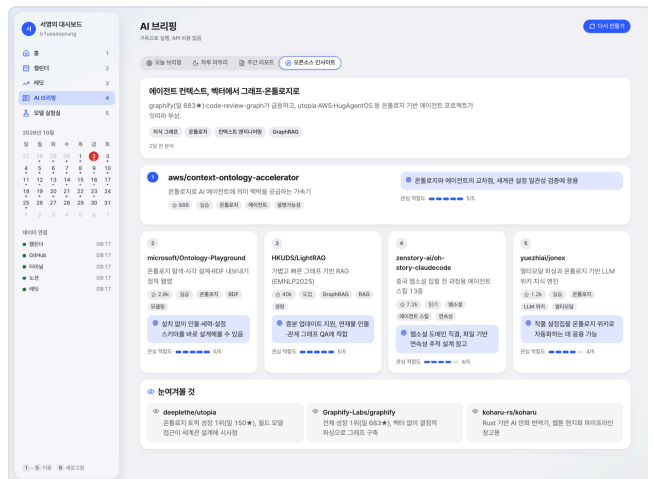
오늘 브리핑



하루 마무리



주간 리포트

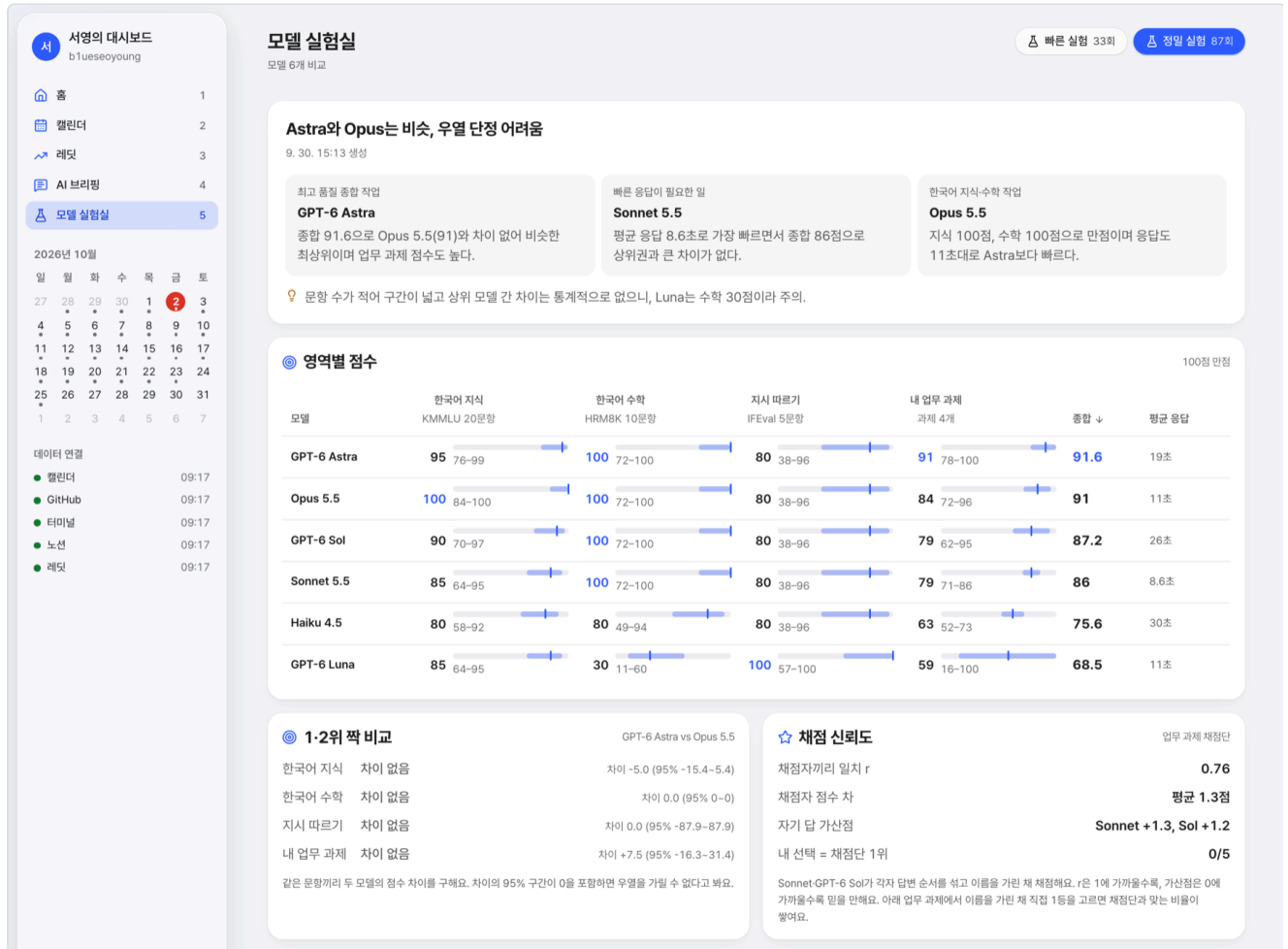


오픈소스 인사이트

- 오늘 브리핑: 한 줄 요약, AI가 본 진행률, 핵심 지표 3개, 조언, 우선순위와 예상 시간이 붙은 할 일, 지금 하고 있는 일. 가장 최근 하루 마무리에서 넘긴 일을 우선순위에 먼저 반영하고, 노선 할 일과 상태로 함께 봄
- 하루 마무리: 아침 계획과 오늘 실제 활동(일정, 노선 할 일, GitHub, 터미널)을 비교한 달성도, 끝낸 일과 근거, 못 끝낸 일과 이유, 내일로 넘길 일(우선순위, 예상 시간), 회고 한 줄. 평일 18:00에 자동으로 만들
- 주간 리포트: 이번 주 커밋, PR·이슈, 일정, 매일의 계획과 마무리를 모아 상사 보고용으로 정리. 요약, 숫자 4개, 주요 성과, 프로젝트별 진행, 다음 주 계획, 이슈·리스크. 금요일 18:30에 자동으로 만들고 `reports/weekly-<일요일 날짜>.md` 로 저장. 마크다운 복사, .md 받기 버튼
- 오픈소스 인사이트: 요즘 흐름과 키워드, 먼저 볼 TOP 5(관심 적합도와 이유), 눈여겨볼 레포 3개
- 결과는 저장돼서 새로고침해도 남음

- 결과가 없을 때, 만드는 중일 때, 실패했을 때, 날짜가 지났을 때 화면이 각각 따로 있음. 오늘 브리핑과 하루 마무리는 날짜가, 주간 리포트는 주가 바뀌면 다시 만들라는 배너가 뜸

모델 실험실



모델 실험실(9월 30일 15:13 정밀 실험 결과, 윗부분)

- 버튼이 두 개다. 빠른 실험(33회 호출)은 내 업무 과제 4개만, 정밀 실험(87회 호출)은 공개 검증 문항 35개까지 더해 돌린다. 버튼에 예상 호출 수가 보인다.
- 모델은 Claude 구독(Opus, Sonnet, Haiku)과 Codex 구독(GPT-6 Astra, Sol, Luna)이다. 양쪽 모두 도구 없이 바로 답하게 맞췄고, 오픈 모델은 넣지 않았다.
- 공개 검증 문항(정밀 실험): KMMLU 20문항(한국어 전문 지식 객관식, 10과목 × 2), HRMBK 10문항(한국어 수학), IFEval 5문항(지시 따르기). 시드 20260930으로 한 번 뽑아 lab-items.json에 고정해서 매번 같은 문항을 푼다. 객관식은 10개, 수학은 5개씩 묶어 한 번에 묻고 정답과 비교해 자동 채점한다. IFEval은 공식 규칙 검사기를 옮겨 엄격 모드로 채점한다.
- 업무 과제 4개(원고 요약, 온톨로지 추출, 업무 메시지 다듬기, 영업일 계산)는 정답이 하나가 아니라서 채점단이 채점한다. 계열이 다른 Sonnet과 GPT-6 Sol이 각자 답변 순서를 섞고 모델 이름을 가진 채 기준으로 1~10점을 매기고, 두 점수를 평균한다.
- 점수표: 모델 × 영역 점수(100점 만점)와 95% 신뢰구간 막대, 종합(영역 점수 평균), 평균 응답 시간. 정답률은 Wilson 구간, 채점 평균은 t 구간이다. 머리글을 누르면 그 기준으로 정렬한다.
- 1·2위 짝 비교: 같은 문항끼리 두 모델의 점수 차이를 구하고, 차이의 95% 구간이 0을 포함하면 "차이 없음"으로 표시한다.

- 채점 신뢰도: 두 채점자의 일치도(r), 평균 점수 차, 자기 답 가산점(채점자가 자기 모델 답을 평소보다 후하게 주는지), 내 선택과 채점단 1위의 일치율.
- 내 선택: 업무 과제를 펼치면 이름과 점수를 가진 답변 A~F가 보인다. 가장 좋은 답을 고르면 공개되고, 고른 결과가 저장돼 채점단을 점검하는 데 쓰인다. "고르지 않고 보기"로 그냥 볼 수도 있다.
- 문항별 결과: 공개 문항은 모델별 맞춤·틀림 표(고른 답과 정답은 칸에 마우스를 올리면 보임), 업무 과제는 답변 원문과 채점자별 점수·이유.
- 방법과 출처: 데이터셋·논문 링크와 라이선스, 채점·통계 방법, 한계(공개 문항 오염 가능성, 적은 문항 수, 채점자도 참가 모델이라는 점).
- 업무 과제·모델·채점단은 `lab.json`, 공개 문항은 `lab-items.json` (`bun lab-items.ts`로 다시 뽑음)에서 바꾼다. 최근 5회 결과를 저장하고, 화면에는 가장 최근 실험이 보인다. 예전 방식(Opus 혼자 채점) 결과는 파일에 남아 있지만 화면에는 나오지 않는다.
- 9월 30일 정밀 실험(15:13, 10분 21초, 호출 오류 0): 종합 GPT-6 Astra 91.6, Opus 91.0, GPT-6 Sol 87.2, Sonnet 86.0, Haiku 75.6, GPT-6 Luna 68.5. 1·2위인 Astra와 Opus는 네 영역 모두 "차이 없음"이라 우열을 가릴 수 없다. 평균 응답은 Opus 11초, Astra 19초이고 가장 빠른 것은 Sonnet(8.6초)이다. 채점자끼리 일치 r은 0.76이고, 두 채점자 모두 자기 모델 답에 1점 남짓 더 줬다(Sonnet +1.3, Sol +1.2). 채점단을 한 계열로만 두지 않는 이유다. 위 캡처가 이 실험의 결과다.
- 같은 날 17:22 빠른 실험(33회 호출, 3분 28초)에서도 업무 과제 1·2위는 Astra(87.5)와 Opus(85)였고 "차이 없음"이었다. 지금 화면에 보이는 가장 최근 실험이 이것이다.
- 정답 키 정정: HRM8K GSM8K#252는 원문이 이틀 합계(73)를 묻지만, 번역은 '오늘' 죽인 수를 물어서 48이 맞다. 강한 모델 5개가 모두 48로 답해서 원문과 대조해 찾았다. 정답을 48로 고치고(`lab-items.ts`에도 기록해 다시 뽑아도 유지), 문항 표에 *로 표시했다. 저장된 답으로 다시 채점하고 결론을 한 번 더 정리해서, 이 실험의 호출은 모두 88회다.

공통

- 사이드바: 메뉴, 이번 달 미니 캘린더, 데이터 연결 상태(캘린더, GitHub, 터미널, 노션, 레딧의 초록·빨강 점과 마지막 갱신 시각)
- 단축키: `1~5` 화면 이동, `R` 새로고침
- 상세 창: 홈의 할 일, 캘린더 일정, 레딧 글을 누르면 화면 옆에서 열림
- 테마: macOS의 라이트·다크 설정을 따름(이 보고서의 캡처는 라이트)
- 반응형: 넓은 화면은 사이드바, 중간 화면은 아이콘 바, 휴대폰은 하단 탭 바
- 상태 미리보기: 주소에 `?state=loading`, `?state=empty`, `?state=error`를 붙이면 그 상태 화면이 열리고, 이때는 저장과 실행이 막힘

터미널 화면 (TUI)

- herdr 같은 터미널 창에서 `bun run tui`로 연다. 탭은 오늘, 레딧, AI 세 개다.
- 같은 서버 데이터를 기호(○ 예정, ● 진행, ● 완료)와 막대로 보여 준다. 오늘 탭 맨 위에 진행률(오늘 노션 할 일 중 완료 비율)이 있다.
- 레딧 탭은 웹 레딧 화면과 같은 데이터(오늘의 흐름, 먼저 볼 TOP 5, 서브레딧별 한국어 제목)를 보여 준다. 새로 모으기는 웹 화면에서 한다.
- 글자색은 터미널 테마의 기본색과 표준 색을 따라서 밝은 배경과 어두운 배경 모두에서 읽힌다.
- 키: `1 2 3` 탭 이동, `j k` 스크롤, `r` 새로고침, `b` 브리핑 생성, `s` 오픈소스 인사이트 생성, `q` 종료
- 노션 할 일도 오늘 탭 일정 목록에 "종일 · 노션"처럼 보인다.

- 하루 마무리, 주간 리포트, 모델 실험실은 웹 화면에만 있다.

4. 사용 흐름

```
bun start → 홈: 오늘 상황 한눈에
├─> 캘린더: 이번 주 일정, 미니 캘린더로 다른 주 보기
├─> 레딧: 오늘의 흐름 → 먼저 볼 TOP 5 → 서브레딧별 글
├─> AI 브리핑: 오늘 브리핑 → 하루 마무리 → 주간 리포트
└─> 모델 실험실: 빠른·정밀 실험 → 영역별 점수와 신뢰구간 → 가린 채 내 선택
```

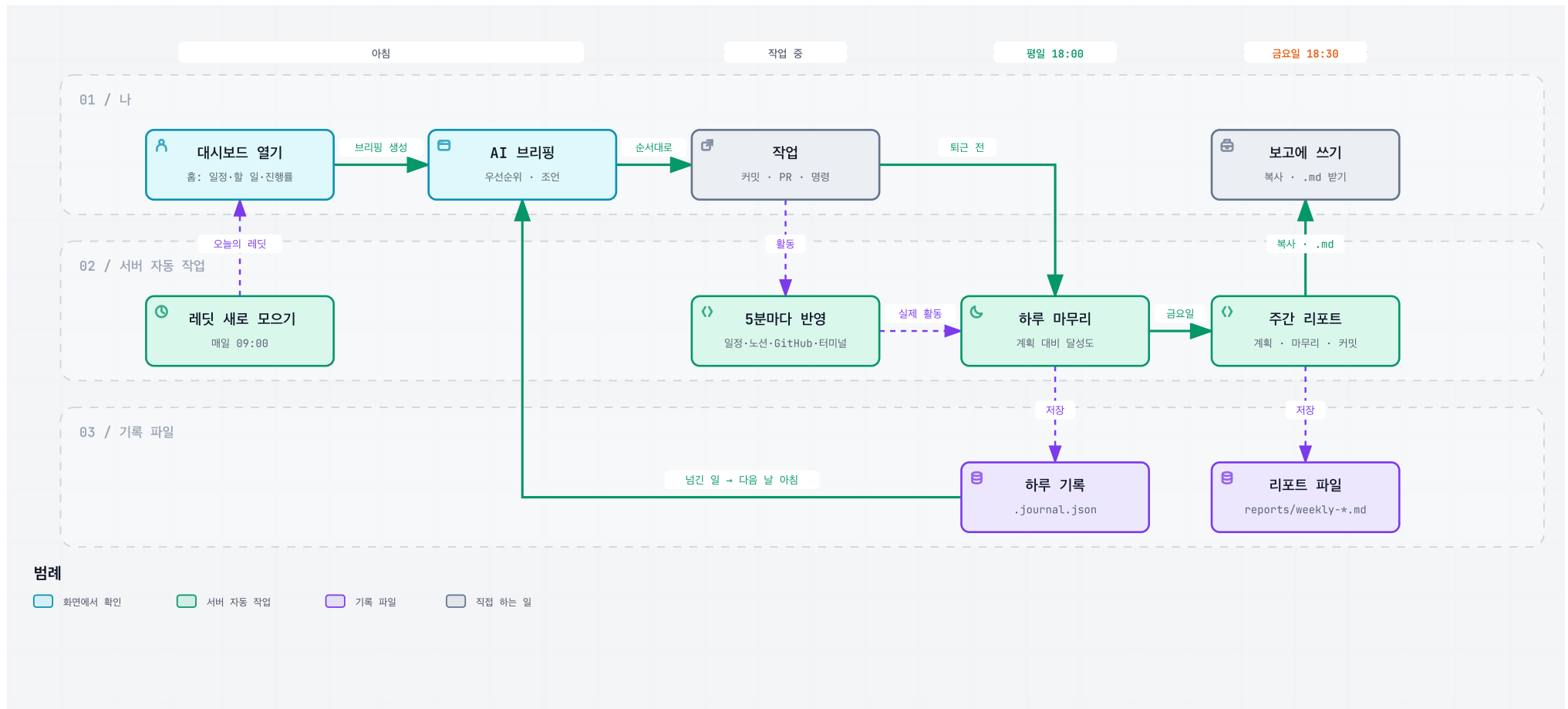
하루와 한 주는 이렇게 흐른다.

1. 아침: 대시보드를 열면 홈에 오늘 일정, 노선 할 일, 진행률이 보인다. 레딧 핫 뉴스는 매일 09:00에 새로 모은다.
2. 작업 중: 커밋, PR, 터미널 명령이 5분마다 반영된다. 타임라인의 "지금" 선으로 남은 일정을 본다.
3. 우선순위가 필요할 때: AI 브리핑을 만들어서 무엇부터 할지와 조언을 받는다. 어제 넘긴 일을 먼저 반영한다.
4. 틈날 때: 레딧 화면에서 오늘의 흐름과 TOP 5로 AI 모델·도구 소식을 한국어로 훑고, 필요한 글만 원문으로 연다.
5. 퇴근 전(평일 18:00): 하루 마무리가 자동으로 만들어진다. 못 끝낸 일은 "내일로 넘길 일"이 되어 다음 날 아침 브리핑으로 이어진다.
6. 금요일 18:30: 이번 주 계획과 마무리, 커밋을 모은 주간 리포트가 만들어지고 파일로 저장된다. 복사하거나 .md로 받아 보고에 쓴다.
7. 모델을 고를 때: 모델 실험실에서 빠른 실험(업무 과제)이나 정밀 실험(공개 검증 문항까지)을 돌려 품질과 속도를 비교한다. 신뢰구간이 겹치거나 짝 비교가 "차이 없음"이면 비슷한 것으로 본다.

자동 생성은 그 시각에 서버가 켜져 있고 맥이 깨어 있을 때만 돈다. 놓치면 AI 브리핑 화면에서 직접 만든다.

하루와 한 주의 흐름은 그림 2에 정리했다.

그림 2 하루와 한 주 흐름



위 칸부터 내가 하는 일, 서버가 자동으로 하는 일, 남는 기록 파일이다. 매일 09:00 레딧 인기 글을 모아 두고, 아침에 대시보드를 열어 AI 브리핑으로 할 일 순서를 정한다. 작업하는 동안 커밋·PR·명령과 노션 할 일이 5분마다 반영되고, 평일 18:00 하루 마무리가 계획과 실재를 비교한다. 못 끝낸 일은 하루 기록(`.journal.json`)을 거쳐 다음 날 아침 브리핑에 먼저 들어가고, 금요일 18:30에는 한 주의 기록을 모아 주간 리포트를 만든다.

5. 데이터 흐름

데이터 출처 → Bun 서버(모으기, 캐시, 09:00 갱신) → 웹 화면과 터미널 화면

데이터	가져오는 방법	갱신
일정	Mac 캘린더 앱(구글 캘린더가 동기화된 것)을 AppleScript로 읽음	5분. 캘린더 화면용 두 달 치는 서버가 켜질 때 미리 읽음(약 40초)
노션 할 일	노션 API로 Task DB에서 '일정'이 있는 할 일을 읽음. 대시보드에서 추가하거나 완료 체크한 것은 노션에 바로 저장. 토큰은 <code>.env</code> 의 <code>NOTION_TOKEN</code>	일정과 같음. 추가·체크는 바로 반영
GitHub 활동	<code>gh</code> CLI로 GitHub 공식 API 호출	5분
터미널	<code>~/.zsh_history</code> 를 읽음. 키, 토큰, 비밀번호처럼 보이는 부분은 가림	5분
레딧	레딧 공식 RSS(<code>/r/<서브레딧>/top/.rss?t=day</code>)를 로그인 없이 읽음. 로그인 없는 요청은 1분에 1번만 허용돼 서브레딧마다 1분 간격. 한국어 제목·요약·순위는 로컬 <code>claude -p (sonnet, 도구 없음) 1회</code> . 결과는 <code>.reddit-cache.json</code>	매일 09:00. 서버가 꺼져 있었으면 다음에 열 때, "새로 가져오기"로도
오픈소스	<code>gh search repos</code> 로 GitHub 공식 검색 API 호출. 오픈소스 인사이트(웹 AI 브리핑, 터미널 화면 AI 탭)에서 씀	매일 09:00. 서버가 꺼져 있었으면 다음에 열 때
AI 결과	로컬 <code>claude -p --effort max</code> . 정해진 JSON 형식으로 받음	만들기 버튼을 누를 때. 하루 마무리는 평일 18:00, 주간 리포트는 금요일 18:30에도 자동
하루 기록	날짜별 아침 브리핑과 하루 마무리를 <code>.journal.json</code> 에 모음. 다음 날 브리핑과 주간 리포트가 읽음	브리핑·마무리를 만들 때
모델 실험	<code>lab.json</code> 의 업무 과제와 <code>lab-items.json</code> 의 공개 문항(KMMLU·HRM8K·IFEval)을 Claude는 <code>claude -p --model</code> , Codex는 <code>codex exec -m</code> 으로 모델마다 보냄. 공개 문항은 정답으로 자동 채점, 업무 과제는 Sonnet·GPT-6 Sol 채점단이 채점. 결과는 <code>.lab-cache.json</code>	실험 버튼을 누를 때

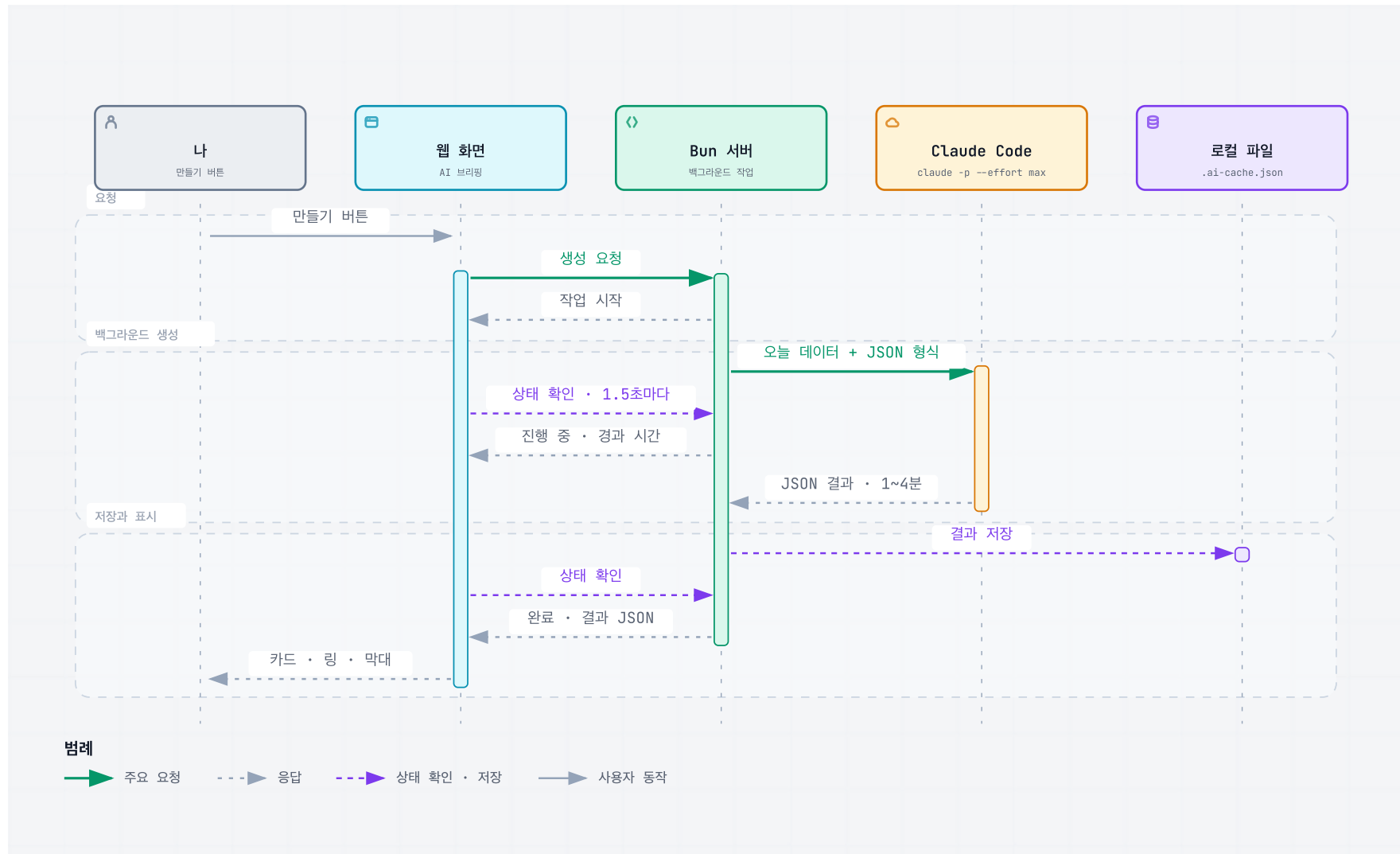
크롤링은 하지 않는다. 모두 공식 API·공식 RSS이거나 이 맥 안에 있는 데이터다.

6. AI 생성 흐름

1. 만들기 버튼을 누르거나 정해진 시각이 되면 서버가 백그라운드 작업을 시작한다.
2. 서버가 오늘 데이터나 오픈소스 목록을 로컬 Claude Code에 넘기고, 정해진 JSON 형식으로 답을 받는다. 1~4분 걸린다.
3. 그동안 화면은 1.5초마다 상태를 확인해서 진행 배너와 경과 시간을 보여 준다. 이전 결과가 있으면 흐리게 남겨 둔다.
4. 끝나면 JSON을 카드, 링, 막대로 그리고 결과를 파일에 저장한다. 실패하면 오류 배너를 띄우고 이전 결과는 그대로 둔다.

AI 결과가 만들어지는 순서는 그림 3에 정리했다.

그림 3 AI 생성 순서



만들기 버튼을 누르면 서버는 바로 '작업 시작'을 돌려주고 백그라운드에서 Claude Code를 부른다. 그동안 화면은 1.5초마다 상태를 확인해 진행 배너와 경과 시간을 보여 준다. 결과가 오면 서버가 파일에 저장하고, 화면이 JSON을 카드·링·막대로 그린다. 실패하면 오류 배너를 띄우고 이전 결과를 그대로 둔다.

모델 실험실도 백그라운드 작업으로 돈다. 단계마다 여섯 모델(Claude 3개, Codex 3개)에 같은 문항을 동시에 보낸다. 정밀 실험은 한국어 지식 → 한국어 수학 → 지시 따르기 → 업무 과제(답변을 받은 뒤 채점단이 채점) → 결론 순서다. 진행 배너에 지금 단계(예: "한국어 지식 · 묶음 2/4 답변 받는 중"), 경과 시간, 호출 수(예: 21/87회)가 보이고, 이전 결과는 흐리게 남겨 둔다. 정밀 실험은 약 10분(9월 30일 실측 10분 21초) 걸리고, 빠른 실험은 업무 과제만 돌아서 더 짧다(실측 3분 28초). 모델이나 채점자 하나가 실패하면 그 칸만 오류로 표시하고 계속한다. 한 단계에서 모든 모델이 답하지 못하면 실험을 멈추고 이전 결과를 그대로 둔다.

7. 실행

```
cd ~/Projects/dev/open-source/dashboard
(cd ui && bun install) # 처음 한 번: 웹 화면 부품 설치
bun start             # 웹 화면: http://127.0.0.1:3000
bun run tui           # 터미널 화면. 서버가 꺼져 있으면 알아서 켜
bun run e2e           # 3940번 포트에 따로 띄워 API와 화면 흐름을 끝까지 확인(약 1분)
bun run e2e -- --write # 노선에 테스트 할 일을 만들고 체크한 뒤 휴지통으로 보내는 것까지 확인
```

`server.ts`를 고친 뒤에는 서버를 다시 켜야 반영된다. 화면 파일(`ui/src/` 아래)은 저장 후 새로고침하면 반영된다.

- 노선 할 일은 `dashboard/.env`에 `NOTION_TOKEN`이 있어야 보인다. `bun start`를 `dashboard` 폴더에서 실행해야 `.env`를 읽는다. 노선 연결 권한은 콘텐츠 읽기, 삽입, 업데이트가 필요하다.
- Codex 실험은 `codex` CLI가 ChatGPT 계정으로 로그인돼 있어야 한다(`codex login status`).
- `bun run e2e`는 서버를 따로 띄우고 레딧 자동 수집을 끈다(`REDDIT_AUTO=0`). 평소 쓰는 서버와 레딧 요청 한도를 나눠 쓰지 않기 위해서다. 잘못된 요청이 수집이나 실험을 시작하지 않고 거절되는지도 확인한다. 10월 2일 최종 코드에서 14개 항목이 모두 통과했다(39초).

8. 파일 구성

파일	역할
<code>server.ts</code>	데이터 수집, 캐시, API, 09:00 갱신, 하루 마무리·주간 리포트 자동 생성, 웹 화면 내보내기
<code>ui/</code>	웹 화면(React + Tailwind + shadcn/ui). 화면은 <code>ui/src/impeccable/</code> , 데이터 불러오기·쓰기·상태 계산은 <code>ui/src/shared/</code> , 글자 규격은 <code>ui/styles/type.css</code>
<code>bunfig.toml</code>	<code>bun start</code> 가 <code>ui/</code> 의 Tailwind 플러그인으로 화면을 묶게 하는 설정
<code>tui.ts</code>	터미널 화면
<code>e2e.ts</code>	API와 화면 흐름 점검(<code>bun run e2e</code>)
<code>interests.json</code>	관심 분야와 가중치(오픈소스 인사이트용)
<code>reddit.json</code>	레딧에서 모을 서브레딧과 한국어 이름
<code>lab.json</code>	모델 실험실의 업무 과제, 모델(Claude·Codex), 채점단과 결론 정리 모델
<code>lab-items.json</code>	정밀 실험의 공개 검증 문항(고정). 출처, 논문, 라이선스 포함

파일	역할
<code>lab-items.ts</code>	공개 문항을 Hugging Face에서 받아 고정 시드로 뽑는 스크립트(<code>bun lab-items.ts</code>)
<code>.env</code>	노션 토큰(<code>NOTION_TOKEN</code>). 공유하거나 커밋하지 않음
<code>reports/</code>	주간 리포트 마크다운(<code>weekly-<월요일 날짜>.md</code>)
<code>DESIGN.md</code>	색, 글꼴, 간격, 상태별 화면 규칙과 버전별 개정 기록

루트의 `index.html`, `app.css`, `app.js` 는 React로 바꾸기 전의 화면 파일이고 지금은 쓰지 않는다. `ui/src/taste/` 와 `ui/src/compare/` 는 1안을 고를 때 쓴 2안과 비교 화면이다.

9. 부록

- [ui-ux-report.md](#): 초기 디자인 결정과 이유, 버전별 개정과 점검 기록
- [visual-report.html](#): 초기 버전별 화면 변화와 캡처
- 그림 1~3: archify로 만든 도식. 그림 파일은 `docs/figures/`, 원본 HTML과 JSON은 `.archify/`
- 이전 판 보고서(9월 30일): `docs/archive/report-0930/`